

Precision Agriculture IoT System: Case Study

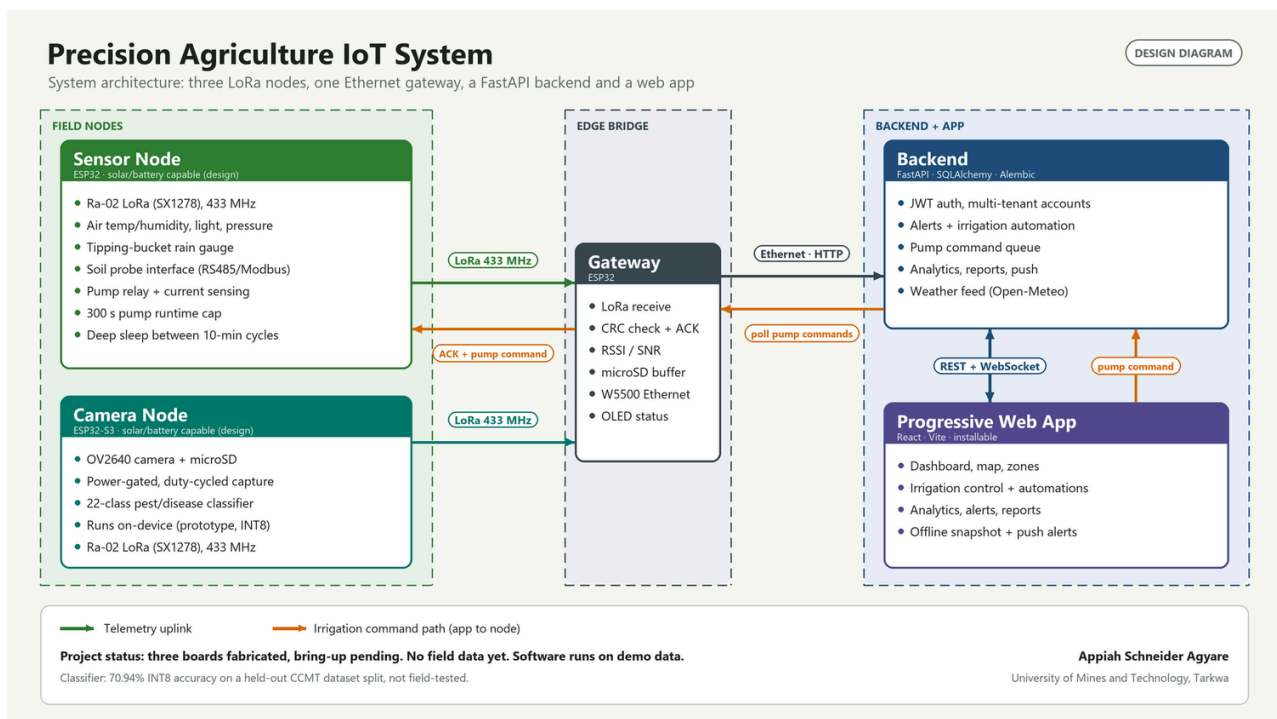
Appiah Schneider Agyare · Cybersecurity student · IoT, embedded systems, networking

Where the project stands: three custom PCBs are fabricated; hardware bring-up and field testing have not happened yet. The software runs on demo data. This case study separates what is designed, implemented and tested from what is still to do.

The problem

Many farms decide when to irrigate, and when to look for pests or disease, by routine and by eye. Continuous soil and weather data is rarely available, and internet-first products struggle where mobile data is unreliable. I wanted to see how far a low-power, long-range, locally buffered design could go, and to learn the full stack, from the circuit board up to the dashboard.

The solution: three nodes and one app



System architecture (design diagram). Boards fabricated, bring-up pending; no field data yet.

- **Sensor Node:** measures air temperature and humidity, light, pressure and rainfall, is designed to read a soil probe, monitors power, and switches an irrigation pump through a relay with a hard runtime limit.
- **Gateway:** receives LoRa packets, checks them, buffers to an SD card, and forwards them to the backend over Ethernet. It is not a sensor node.
- **Camera Node:** captures crop images and runs a small on-device image classifier prototype.
- **Web app:** a Progressive Web App with dashboards, map, alerts, irrigation automation, analytics and reports.

My role

I designed and specified the system, drew and reviewed the three PCBs in KiCad, wrote and

structured the firmware and app with AI assistance, managed the component sourcing and manufacturer communication, and verified the work. This was my first electrical engineering project; my background is cybersecurity.

What was built

Area	Status
3 KiCad PCB designs, DRC/ERC documented	Designed and design-verified
Boards fabricated and assembled by a supplier	Done. Bring-up pending.
Firmware for all three boards (ESP-IDF)	Implemented, builds. Not yet validated on hardware.
Backend (FastAPI)	Implemented and deployed. 135 automated tests pass.
Web app (React PWA, 22 pages)	Implemented. Runs on demo data.
Enclosures (OpenSCAD CAD)	Designed. Not printed.
Solar power	Designed. Not tested.
Field deployment	Not done

Design notes

PCBs. The boards are carrier boards: an ESP32 development board and off-the-shelf modules (LoRa radio, RS485 transceiver, Ethernet, camera) plug into headers, and the board adds power routing, protection, connectors and the pump relay driver. Verification used KiCad's DRC and ERC, plus scripted pad-level checks and item-by-item comparison of results before and after each change.

Communication. LoRa at 433 MHz with a simple framed packet and acknowledgement. The acknowledgement also carries irrigation commands back to the Sensor Node, so the node needs only one receive window. No range has been measured yet.

Irrigation safety. The pump runtime is capped at 300 seconds in both the firmware and the backend, and automation adds a cooldown and a weather-aware rain lockout.

Image classifier. A MobileNetV2 model quantized to INT8 classifies 22 crop pest and disease classes from the CCMT dataset (Ghana-sourced, CC BY 4.0). On 3,768 held-out images it reaches 70.94% (float model 80.18%). Results differ a lot by class, and it has not been tested on real field images. I treat it as a prototype, not a diagnostic tool.

Key challenges

1. **Footprints that pass DRC but do not fit the real module.** Row spacing on the LoRa, Ethernet and RS485 module footprints was wrong. DRC could not see it. I corrected each against supplier drawings and datasheets, measured actual pad coordinates, and re-verified.
2. **Header spacing on the camera board.** The two rows for the ESP32-S3 were 30 mm apart instead of 25.4 mm. Correcting it meant re-routing 14 nets with an autorouter; the corrected design passes KiCad's design-rule check with no new violations compared with the original. It applies to future orders, not the boards already made.
3. **A fix that created a short.** Widening a fuse holder footprint made a pad touch a nearby trace. DRC caught it, and I learned to compare violations one by one rather than by count.
4. **Schematic and PCB out of sync.** A forensic re-audit found two diodes wired backwards. Both files now agree.
5. **Firmware pins that disagreed with the copper.** The camera clock was assigned to the wrong GPIO in firmware; two independent sources caught it before it reached hardware.

What I learned

- Tool output is not verification. The physical module is the reference.
- Measure the right thing, and check the checker.
- Compare before and after, item by item.
- Say clearly what is untested. Credibility matters more than impressive claims.
- Reviewing your own system honestly finds real problems. Mine had two.

Security: honest status

I reviewed my own deployed system the way an attacker would, and found two real problems: the Gateway endpoints accepted anonymous requests, and the live-dashboard WebSocket sent every user's data to any client. I fixed both the same day with per-account device keys and an authenticated WebSocket, wrote 34 security tests (and confirmed they fail when I deliberately remove two of the protections), and re-checked the live system.

Still open: the LoRa radio link has a CRC but no encryption or authentication, there is no firmware signing, and the Gateway's HTTPS path is compile-verified but has not run on hardware.

How AI was used

AI assistants helped with review checklists, scripted checks, code and documentation drafting. Every engineering claim was checked against datasheets, supplier drawings, KiCad's own tools or the physical modules. Several AI-generated claims were wrong and were caught that way.

Next steps

Board bring-up and power measurements; soil probe datasheet and calibration; LoRa range and reliability tests; pump and solar/battery tests; authenticated LoRa payloads and firmware signing; a hardware test of the Gateway's HTTPS path; classifier validation on real field images; printing and weather-testing the enclosures. Real results will be added as they happen.

Data source

Mensah, P. K. et al., "Dataset for Crop Pest and Disease Detection", Mendeley Data, DOI 10.17632/bwh3zbpkpv.1 (CC BY 4.0).